Delete my account: how it works, step by step

What Koios keeps about a person, where, and what happens — in order, with the code that does each part — when they delete their account from Settings.

Written 18 October 2026, for pull request #301. Code references are to the repository at that commit. The screenshots are of the real dialog, in the Koios theme.

Contents

1. What Koios keeps, and where
2. Where the option is
3. The dialog, before anything is deleted
4. Pressing Delete: the page's steps, in order
5. The server: proving who is asking
6. The server: deleting from every store
7. Shared projects: finding them three ways
8. When something fails
9. What is deliberately not deleted
10. Security notes
11. Tests, and how to run them
12. Appendix: the API and the files

1. What Koios keeps, and where

Koios has no database of its own and no account to create: signing in with Google is the account. Even so, using the site leaves records in three places, and each has its own owner and its own way of being deleted.

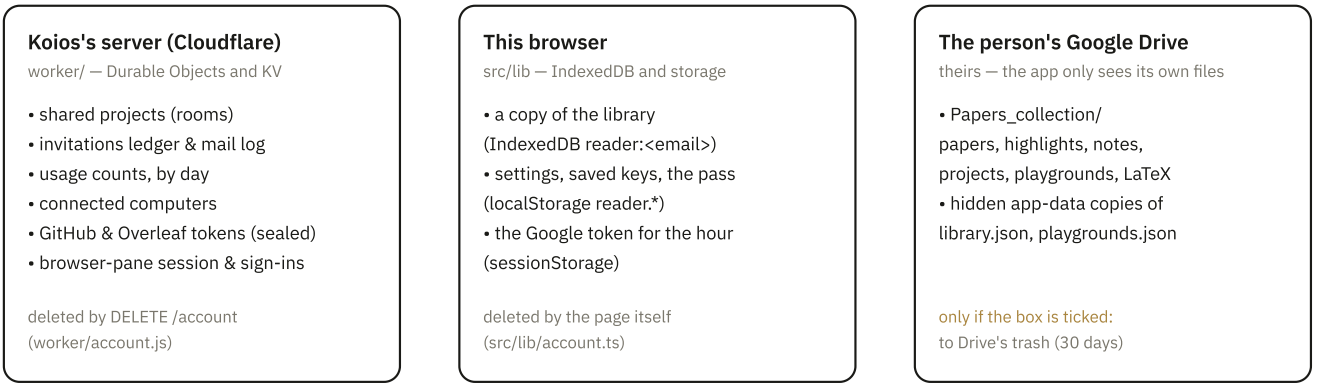


Figure 1. The three places a person's records live. The server's part is deleted by one request; the browser's by the page; Drive only on request, and reversibly.

STORE	NAMED BY	WHAT IS KEPT ABOUT A PERSON	CODE
Shared projects	one object per room (`room:<id>`)	the room's owner; a member record (email, role, name, picture); invitations by email; "last seen"; and the logbook, where each note, link and stroke says who wrote it (`by`) and meeting notes list attendees	`worker/projectRoom.js`, `worker/logbook.js`

Invitations ledger	one object for the site	every invitation sent (by whom, to whom, whether taken up), personal links and who joined from them, which rooms each account is in (`rooms:<email>`), and the site's mail log	`worker/invites.js`
Usage tally	one object for the site	per day, per email: sign-ins, searches, PDFs fetched, AI calls and their cost — the owner's Usage page	`worker/usage.js`
Computers	one object per email (`devices:<email>`)	each Koios Companion the account connected: name, hardware, its address and Jupyter token	`worker/devices.js`
GitHub, Overleaf	KV keys, the email hashed	the tokens, sealed with `SECRETS_KEY`; when added and last used	`worker/github.js`, `worker/latex.js`, `worker/secrets.js`
Browser pane	per browser (client id), not per email	the open session, a short log of pages opened, sites that showed a captcha; and in KV the sign-in cookies the pane kept	`worker/browserSession.js`, `worker/browse.js`
The pass	—	**nothing**: the 30-day pass is signed, not stored (`server/passes.js`). It stops working only when it expires.	`server/passes.js`

2. Where the option is

Signed in, **Settings → Account** shows the Google account card. Under the account row is a quiet line with the one button that starts the deletion. It is deliberately not red and not large: it is there when wanted, and nowhere near Sign out.

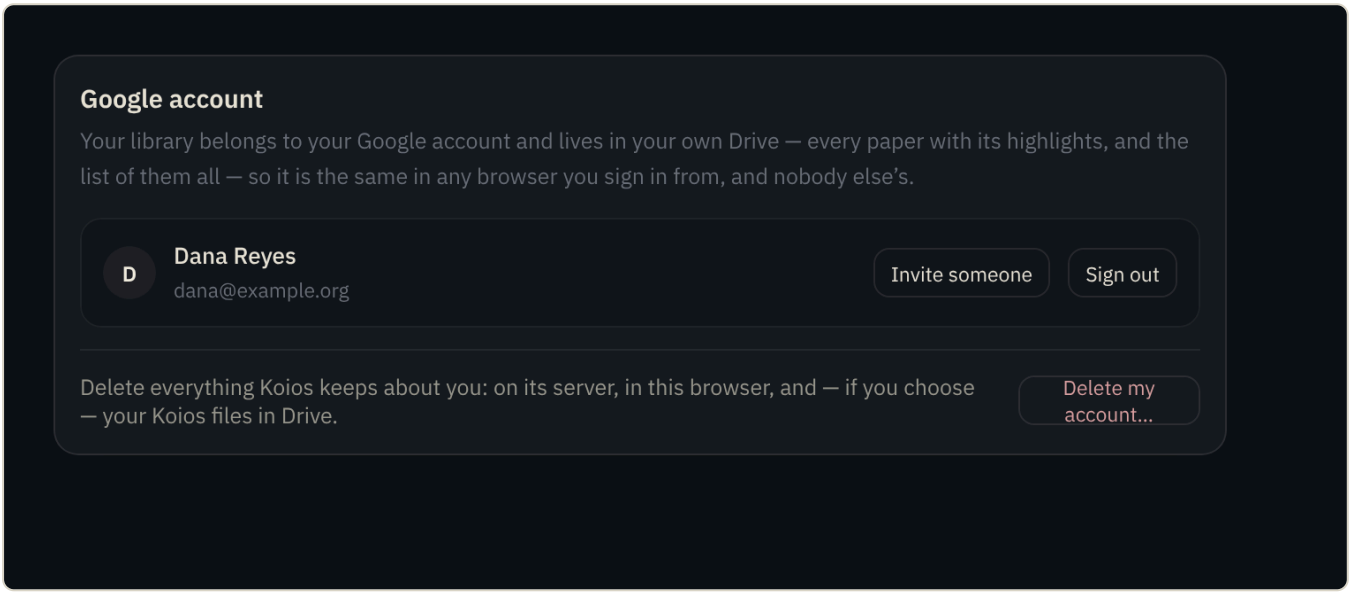


Figure 2. The account card (`src/components/Settings.tsx`, the `set-danger` row). The button opens the dialog; nothing happens yet.

3. The dialog, before anything is deleted

The dialog (`src/components/DeleteAccount.tsx`) is built so that the person knows exactly what will go before they can press anything. Opening it does one read and no writes.

1. **It asks the server what it keeps**
   On opening, the page calls `readAccount()` (`src/lib/account.ts`), which is `GET /account` with the person's pass. The server answers with a summary (`accountSummary` in `worker/account.js`): the shared projects they own and the ones they joined (by name), invitations waiting, computers connected, days of usage counted, invitations sent, and whether a GitHub or Overleaf connection exists. Nothing is changed by this call.

2. **It lists what will go, under three heads**
   On Koios's server — one line per thing the summary found, with project names in quotes so the person can recognise them. **In this browser** — always the same: the library copy, settings, saved keys; signed out; Google's access revoked. **In your Google Drive** — a checkbox, **off by default**.

3. **It asks for the address to be typed**
   The Delete button stays disabled until the person types their own email address exactly (case does not matter). Cancel is always available; clicking outside the dialog closes it too, except while deleting.

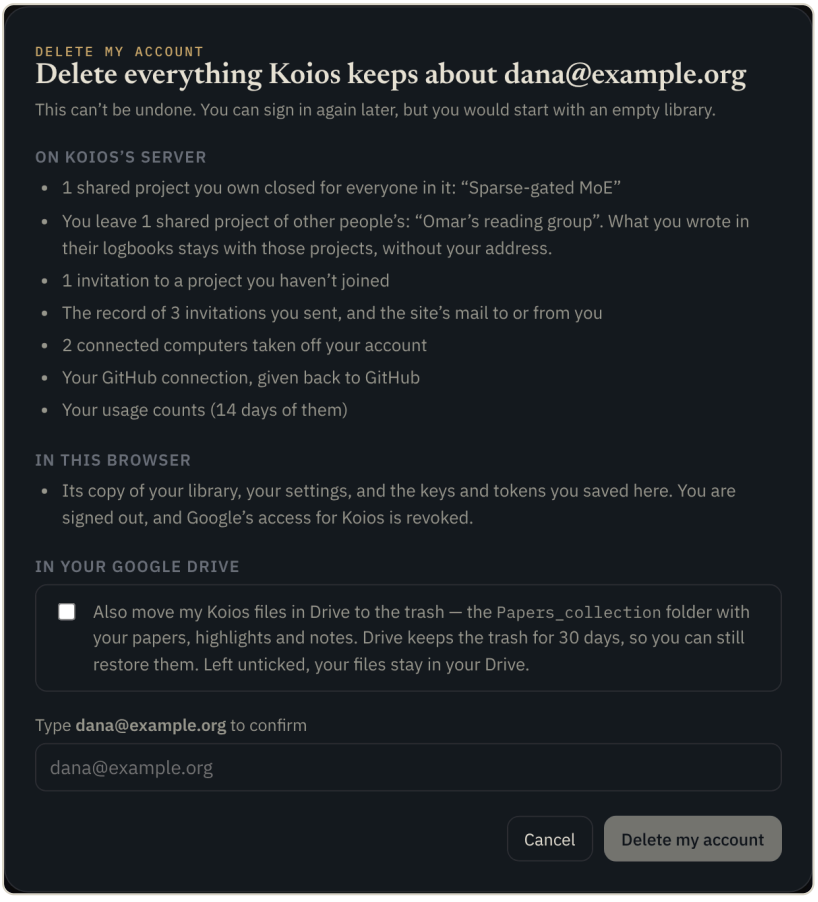


Figure 3. The dialog as it opens, for an account that owns one shared project, is in another, has one invitation waiting, two computers and a GitHub connection. The Drive box is unticked and the button disabled.

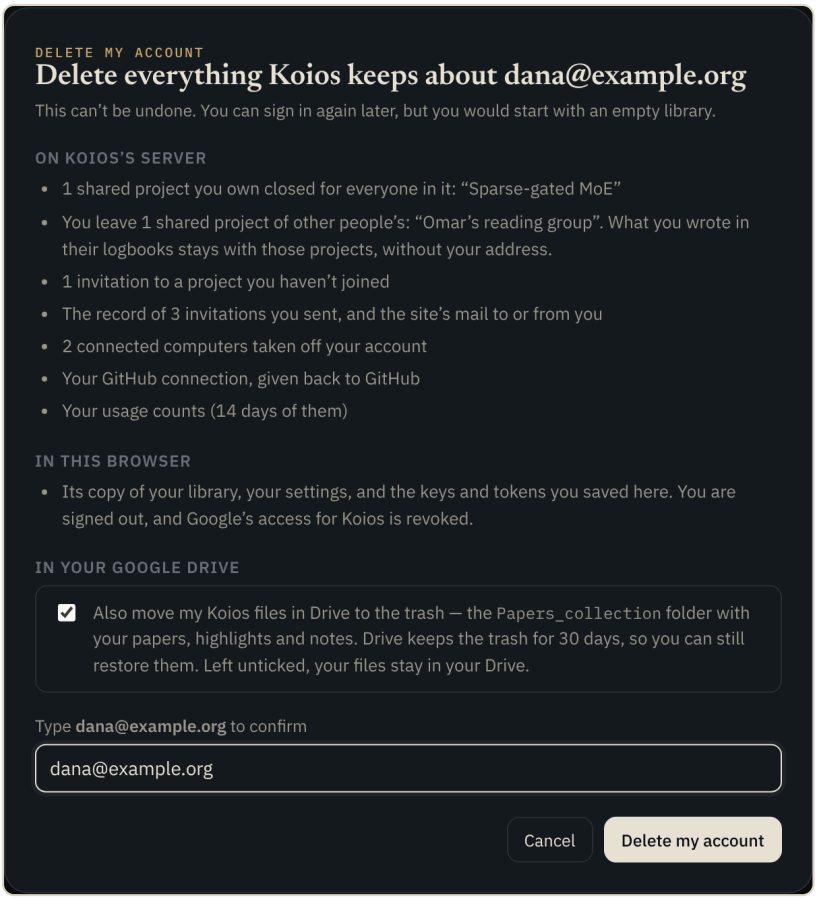


Figure 4. The Drive box ticked and the address typed: the button is live. Only now can anything be deleted.

> **Three states the summary can be in.** While it loads: "Looking up what is kept…". If the server cannot be asked (no pass, or no server in this deployment): a line saying nothing of theirs is kept there, and deleting only does the browser's part. If the server answered with an error: the error is shown, and deleting still removes all of it.

4. Pressing Delete: the page's steps, in order

Everything from here is `deleteAccount()` in `src/lib/account.ts`. The order matters: Drive needs the Google token that signing out revokes, so Drive goes first; the server needs the pass and the token, so it goes before the browser forgets them; the browser's own copy goes last, and only then does the page sign out and start again.

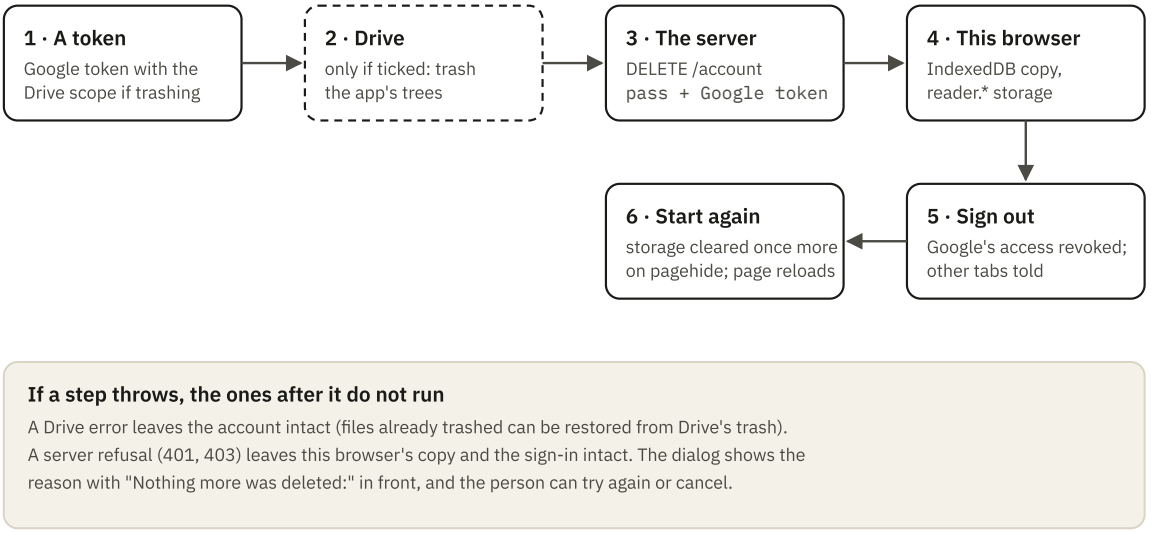


Figure 5. The page's steps. Step 2 is dashed because it runs only when the Drive box was ticked.

1 Get a Google token

If the Drive box is ticked, `ensureDriveToken(clientId)` is used: it returns the live token when it already carries the Drive scope, and otherwise renews quietly on the existing grant, throwing if Drive was never granted. If the box is not ticked, any live token will do (`liveAccessToken()`), with the same renewal as a fallback. The token is needed in either case, because the server will ask for it in step 3.

2 Drive, only if ticked: `trashDrive(token)`

With the `drive.file` scope, a listing of Drive returns *only files the app itself created*, wherever the person has since moved them. The page lists them all (`fields=id,parents,ownedByMe`, following pages), then finds the **roots**: files whose parent is not itself one of the app's files. Trashing a root takes its whole tree with it, so `Papers_collection` goes in one request even if it holds hundreds of papers. Files the person does not own (a collaborator's folder shared into their Drive) are skipped: they are not theirs to trash, and trying would fail and stop the deletion.

Then, if the token has the `drive.appdata` scope, the hidden copies (`library.json`, `playgrounds.json` in the app-data space) are deleted outright: Drive has no trash for app-data files.

Drive's trash keeps files for 30 days, and the person can restore anything from it themselves.

3. **The server:** `DELETE /account`
   One request, with three things on it: the pass in `Authorization`, the Google token in `X-Google-Token`, and a JSON body `{ "rooms": [...] }` listing the room ids of every shared project this browser's library knows (see §7). The browser's client id goes along as always, in `X-Reader-Client`, so the server can also clear what it keeps by browser. If the answer is not OK, the page throws here with the server's reason, and nothing below runs.

4. **This browser's copy:** `deleteLibraries(email, andGuest)`
   The app's open IndexedDB connection is closed, then the database `reader:<email>` is deleted (`src/lib/db.ts`). If the "guest" database — the one a browser uses before anyone signs in — was claimed by this account when it first signed in, it is deleted too; otherwise it is left, since its papers were never this account's. Then every `localStorage` and `sessionStorage` key beginning `reader.` is removed: settings (which hold the pass), saved AI keys, layout, the Google session for the hour.

5. **Sign out:** `google.signOut()`
   The Google access token is revoked with Google, the desktop app (if the page is in it) hands back the grant it kept, and the other tabs of the site are told to sign out and reload.

6. **Start again**
   A `pagehide` listener clears the `reader.*` keys a second time as the page unloads, so anything the app wrote back in the last moment goes too. The page then navigates to the site's root, signed out, with an empty guest library.

---

5. The server: proving who is asking

The route is in `worker/index.js` (`/account`). A pass alone would be a weak basis for deleting someone: it lasts 30 days, and a pass that leaked — from a shared computer, say — should not be enough to destroy an account. So deleting needs two proofs that agree.

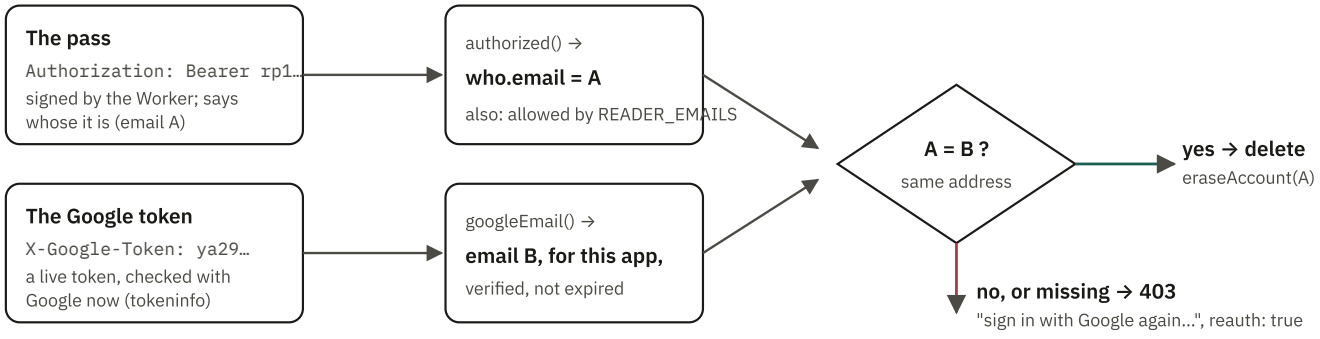


Figure 6. Two proofs that must name the same account. The Google token is checked with Google at request time, never trusted from its contents.

1. **Origin.** The request must come from the site itself (`ALLOWED_ORIGINS`); anything else is refused with 403 before any work.

2. **The pass.** `authorized()` reads and verifies it. No pass, or a pass for an address that `READER_EMAILS` no longer allows: 401. The site owner's own secret (`READER_TOKEN`) names no email, so it cannot delete anyone either.
3. **The Google token.** `googleEmail()` (`server/passes.js`) sends it to Google's `tokeninfo` endpoint and accepts the answer only if the token was issued to this app's client id, the address is verified, and the token has not expired. A token from another site, or an old one, yields nothing.
4. **They must agree.** If the two addresses differ, or the token check failed: 403 with `reauth: true`, and nothing is deleted. The dialog shows the message and the person can sign in again and retry.

`GET /account` — the summary — needs only the pass: it reveals nothing that the person's own pages do not already show, and changes nothing.

---

6. The server: deleting from every store

`eraseAccount(env, email, { client, rooms, browserSession })` in `worker/account.js` visits each store in turn. Each store has one small internal route for forgetting a person; none of those routes is reachable from the internet (§10). A store that is not bound in this deployment, or that does not answer, is skipped and named in the answer's `skipped` list, so the rest still happens.

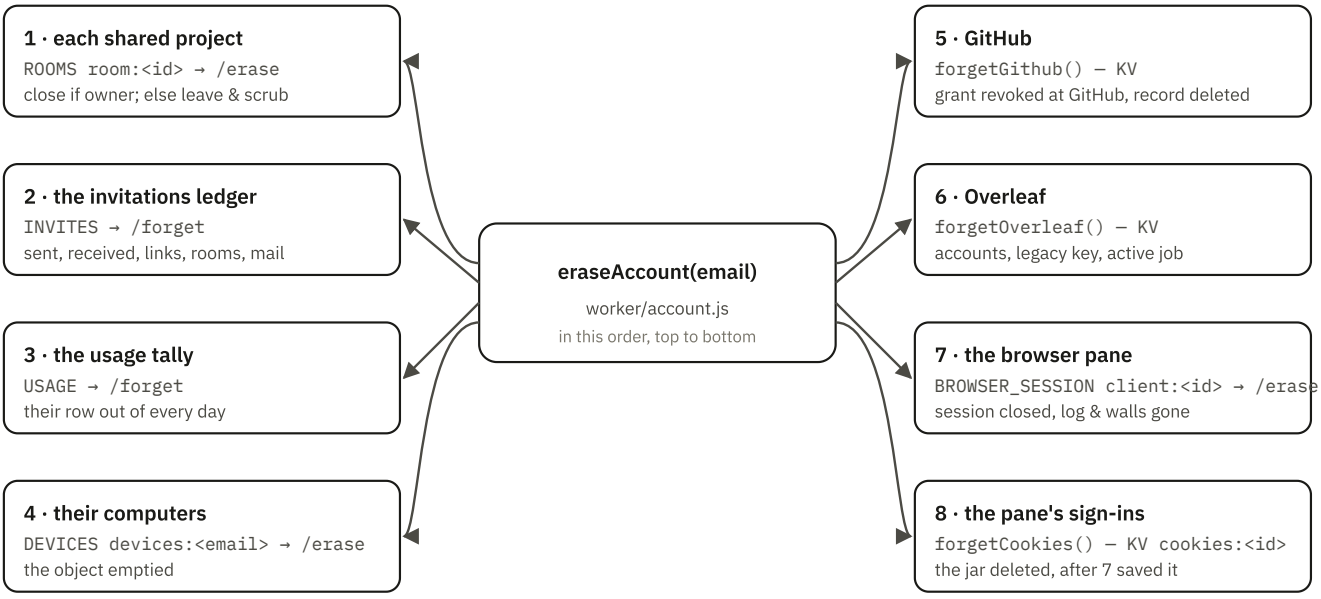


Figure 7. One function, eight stores. Steps 7 and 8 are by browser rather than by account, so they use the client id the request carried.

6.1 Shared projects (`worker/projectRoom.js`, route `/erase`)

For each room id (from the ledger, the inbox of invitations, and the list the page sent), the room object is asked to erase the person. What it does depends on their part in it:

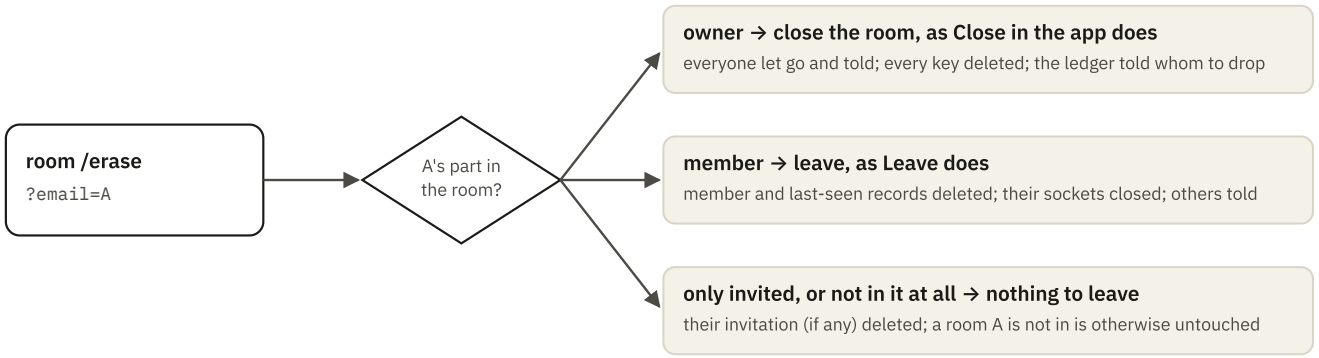


Figure 8. Then, in every case but a closed room, the scrub: A's address comes off what stays.

The scrub. What the person wrote in someone else's logbook is that project's record of its meetings and links, so it stays — but their address does not. Every note, link, stroke and the shared pad whose `by` is their address gets `by: ''`; meeting notes drop their address from `attendees`; invitations they sent to others keep the invitation but lose the `by`; and the room's `updatedBy` is blanked if it was theirs. After this, the room's storage contains the address nowhere (the test checks the whole object for it).

When a room was closed, the Worker also tells the ledger (`/room-closed`) with the list of members, so each of them is taken off that room on their own `rooms:` record, as happens when an owner closes a room by hand.

6.2 The invitations ledger (`worker/invites.js`, route `/forget`)

- Every event the person sent or was sent is deleted — project invitations, "Invite someone" mails, and their personal links (with the `link:<token>` index, so the link stops resolving).
- Where they merely joined from *someone else's* personal link, they are removed from that link's `joined` list; the link and its owner's tally stay.
- Their inbox index (`to:<email>`) and their room list (`rooms:<email>`) are deleted.
- Mail-log entries to or from them are deleted. The mail provider's own delivery log is outside Koios (§9).

6.3 The usage tally (`worker/usage.js`, route `/forget`)

Counts are kept per day, as one record per day keyed by email. The person's key is removed from every day's record; a day left with nobody in it is deleted. The owner's Usage page simply no longer has the row. (The same object answers `/mine` with how many days have counts, for the dialog's summary.)

6.4 Computers (`worker/devices.js`, route `/erase`)

The account's object — one per email — is emptied with `deleteAll()`. A Companion still running on one of those computers will find, next time it checks in, that its row is gone and its secret no longer matches; it does nothing further until someone claims it again.

6.5 GitHub and Overleaf (`forgetGithub`, `forgetOverleaf`)

These are the same functions the Disconnect buttons use. GitHub: the person's grant to the app is revoked with GitHub first (for sign-in and GitHub-App connections), then the sealed record is deleted from KV. Overleaf: every account's sealed tokens are deleted, the pre-`secrets.js` legacy key with them, and the marker of a compile in progress. The KV keys are an HMAC of the email, never the email itself, which is why `secretsKey()` is needed to find them.

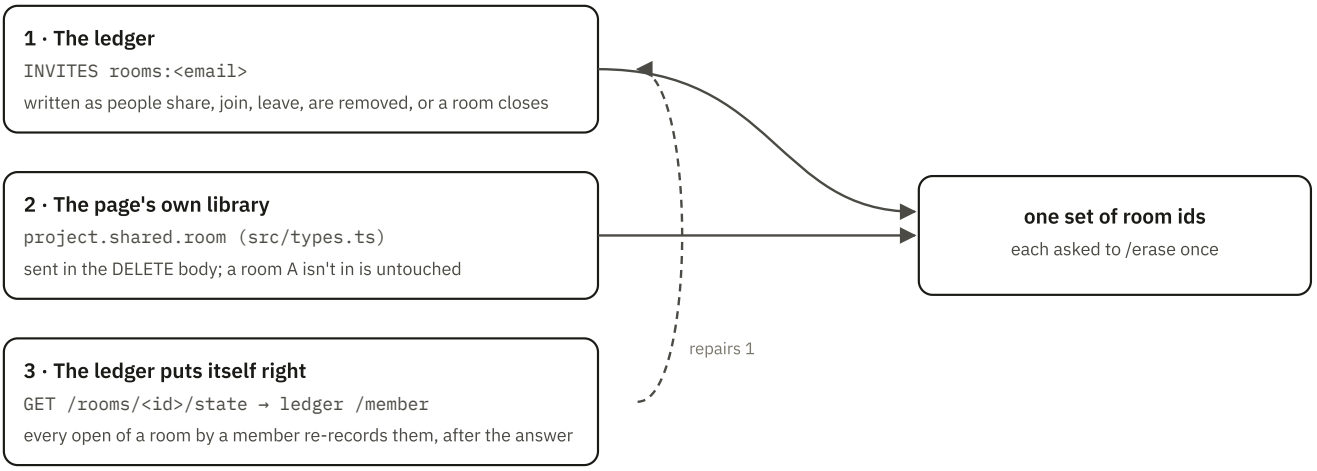
6.6 The browser pane and its sign-ins

These are kept by *browser* (the client id in `X-Reader-Client`), not by account, because the pane is the browser's. The session object for this client is told to release any open browser and `deleteAll()` — the session, its short log of pages opened, and the hosts it remembered as having shown a captcha. Releasing saves the pane's cookies to the jar; so the jar (`cookies:<client>` in KV) is deleted straight after, by `forgetCookies()`, the same function as "Forget sign-ins".

---

7. Shared projects: finding them three ways

A room can only be reached by its id, so the deletion needs a list of the rooms the person is in. One index would be a single point of failure — a record lost means a room the deletion never sees — so there are three, and a room is missed only if all three lose it.



The ledger and the rooms were added the same day (14 Oct 2026), so there is no "before the ledger" to catch up on; this is insurance against drift.

Figure 9. Three sources, one set. The server-side list and the client-side list are merged and de-duplicated before any room is touched.

Why this is safe even with a wrong id from the page: the room's `/erase` takes the caller's email from the Worker (not from the page), and on a room where that email is neither owner, member nor invitee it deletes an invitation that does not exist and scrubs nothing — a no-op. So the page can only ever cause its own account to be removed from rooms it is actually in.

---

8. When something fails

The page deletes in an order such that a failure leaves things recoverable, and it says so plainly.

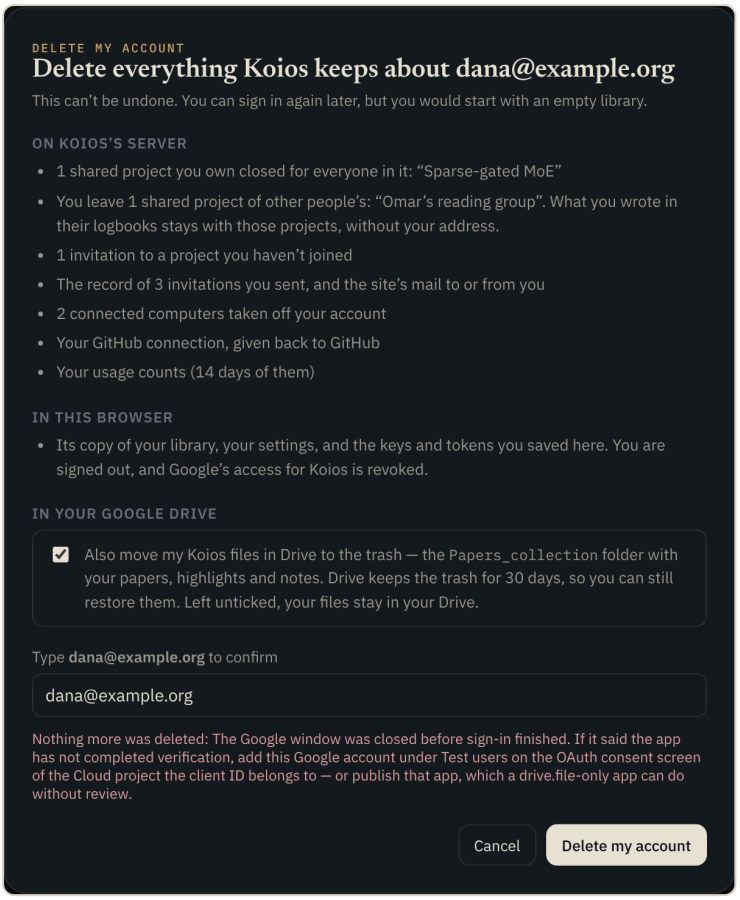


Figure 10. A step failed — here, the Google window that renews the token was closed before it finished. The dialog keeps the person's choices, says "Nothing more was deleted" with the reason, and the button is live again.

WHERE IT FAILS	WHAT HAS HAPPENED BY THEN	WHAT THE PERSON SEES AND CAN DO
Getting the token (step 1)	nothing	Google's reason (window closed, Drive not granted); retry, or untick Drive
Drive (step 2)	some files may be in Drive's trash — restorable there for 30 days	"Drive request failed (…)"; retry, or untick Drive and the account still deletes
The server (step 3): 403 `reauth`	Drive, if ticked	"sign in with Google again to delete this account"; sign in and retry
The server: 401	Drive, if ticked	the pass has expired or the address is no longer allowed; sign in again
The server: a store did not answer	the server deletes the rest and answers OK, naming the store in `skipped`	the deletion completes; the skipped store's record stays until the owner removes it
Deleting the browser's copy (step 4)	everything on the server; Drive, if ticked	the page still signs out and reloads; a database another tab held open is deleted as soon as that tab closes

9. What is deliberately not deleted

- **The person's Drive files, unless they ticked the box.** They are the person's own files in their own Drive, and Koios has no claim on them. Ticked, they go to Drive's trash, not oblivion.
- **Other people's copies of a shared project.** A project is mirrored into each member's own library. Deleting the owner's account closes the room, but what each member already has in their Drive stays theirs.
- **What they wrote in others' logbooks.** The notes, links and strokes stay with that project (without the address): they are the project's record of its meetings, and other members rely on them.
- **The mail provider's delivery log.** When the site emailed an invitation on the person's behalf, the provider (Brevo, SendGrid, ...) keeps its own record of that delivery for a time under its own policy. Koios's own log of it is deleted.
- **Their Google account**, obviously. Koios's access to it is revoked; the person can also check at `myaccount.google.com/permissions`.
- **Rate-limit counters** keyed by email are 60-second windows, not a store; they expire on their own.

> **Idempotent.** Deleting twice is safe: the second summary finds nothing, and every forgetting route tolerates an absent record. The test "deleting twice finds nothing the second time" checks this.

10. Security notes

- **Two proofs** (§5): a leaked 30-day pass alone cannot delete an account; a Google token alone (no pass) cannot either.
- **Internal routes are not public.** The forgetting routes (`/erase`, `/forget`) exist only inside the Worker's objects. The public passthroughs forward only named actions: `/rooms/<id>/…` an allow-list without `erase`; `/devices/…` only `claim` and `forget`; and `/browse/…`, which forwards most paths to the pane's object, now refuses `erase` with 404 before forwarding (the test script checks this). `/account` itself is the only way in, and it needs both proofs.
- **Origin-bound.** `/account` answers only requests from the site's own origins, like sign-in and sharing.
- **No enumeration.** The summary is only ever about the account whose pass is presented; there is no way to ask about someone else.
- **The page can only harm itself.** The room ids it sends are acted on under the caller's own email, and a room the caller is not in is untouched (§7).
- **Secrets are never read to delete them.** GitHub and Overleaf records are found by the HMAC of the email and deleted sealed; the GitHub token is opened only to hand the grant back to GitHub, as Disconnect does.

11. Tests, and how to run them

`scripts/account.test.mjs` runs the real Worker and the real Durable Object classes against in-memory stand-ins for Cloudflare's storage and KV, with Google's `tokeninfo` stubbed so a token `google:<email>` is that account's. It is part of `npm run test:api`.

```
node --test scripts/account.test.mjs        # just these
npm run test:api                           # all 532 server tests, these among them
```

CASE	WHAT IT PROVES
says first what is kept about the account	the summary counts rooms owned and joined by name, computers, days of usage, invitations sent, GitHub
takes a Google token for the same account, not a pass alone	no token → 403; the other person's token → 403; no pass → 401; both, agreeing → 200
then leaves nothing of the account, and the other person's things as they were	after deleting one of two people who share projects with each other: her room is closed for him and off his list; his room goes on without her, her note stays with `by: ''` and her address gone from its attendees; the whole storage of his room, the ledger, the tally and KV contain her address nowhere; his counts and his GitHub record are intact; her computers' object is empty
sees to the rooms the page sends, when the ledger has lost them	with her ledger record deleted and the ids sent in the body (plus a junk one), the same closing and leaving happens
is told of a room again by the ledger each time a member opens it	a lost record comes back on the next `/state`; a room not opened stays off the list until it is
then, deleting twice, finds nothing the second time	idempotence

What the tests do not cover, because it needs a real browser signed in to Google: the Drive trashing and the page's own clean-up (§4, steps 2 and 4–6). Those were reviewed by hand; a click-through with a spare Google account before relying on the feature is still recommended.

12. Appendix: the API and the files

GET /account

```
Authorization: Bearer <pass>
→ 200
{
  "email": "dana@example.org",
  "owned":  [{ "room": "<32 hex>", "project": "Sparse-gated MoE" }],
  "joined": [{ "room": "<32 hex>", "project": "Omar’s reading group" }],
  "invited": 1, "devices": 2, "usageDays": 14, "invitationsSent": 3,
  "github": true, "overleaf": false
}
```

DELETE /account

```
Authorization: Bearer <pass>
X-Google-Token: <a live Google access token for the same account>
X-Reader-Client: <this browser's client id>
Content-Type: application/json
{ "rooms": ["<32 hex>", …] }          # the shared projects this library knows; optional

→ 200  { "ok": true, "closed": 1, "left": 1, "invitations": 4, "mail": 2,
         "usageDays": 14, "devices": 2, "github": true, "overleaf": false,
         "browser": true, "skipped": [] }
→ 401  no pass, or an address no longer allowed
→ 403  not from the site; or the Google token is missing, invalid, or for another account
       { "error": "sign in with Google again to delete this account", "reauth": true }
```

Internal routes (Worker → its own objects; not reachable from outside)

OBJECT	ROUTE	DOES
`ProjectRoom`	`POST /erase?email=`	close (owner) / leave (member) / drop invitation; scrub the address from the logbook, invitations sent, `updatedBy`. Answers `{ ok, members, log }` when closed, else `{ ok, left }`
`Invites`	`POST /forget { email }`	events by or to them, their links, `to:`, `rooms:`, mail entries; off others' link `joined` lists
`Usage`	`POST /forget { email }` · `GET /mine?email=`	their key out of every day / how many days have it
`Devices`	`POST /erase`	`deleteAll()` on the account's object
`BrowserSession`	`POST /erase?client=`	release the browser, `deleteAll()`

Files

FILE	PART
`src/components/Settings.tsx`	the "Delete my account…" row on the account card; hands the dialog the room ids from the library
`src/components/DeleteAccount.tsx`	the dialog: summary, Drive box, typed confirmation, errors
`src/lib/account.ts`	`readAccount()`, `deleteAccount()`: Drive, the request, the browser's clean-up, sign-out
`src/lib/db.ts`	`deleteLibraries()`: the IndexedDB copies
`worker/index.js`	the `/account` route and its two proofs; `/rooms/<id>/state` re-recording membership; `/browse/erase` refused
`worker/account.js`	`accountSummary()`, `eraseAccount()`
`worker/projectRoom.js`	the room's `/erase`

`worker/invites.js`	the ledger's `/forget`
`worker/usage.js`	`/forget`, `/mine`
`worker/devices.js`	`/erase`
`worker/browserSession.js`	`/erase`
`worker/github.js, worker/latex.js`	`forgetGithub()`, `forgetOverleaf()`, shared with the Disconnect routes
`public/privacy/index.html`	"Deleting your data" describes all of this to the public
`scripts/account.test.mjs`	the six tests above